MAILDESK — EMAIL MANAGEMENT SYSTEM

Integrated Object-Oriented Programming and Database Management Systems Project Report

Submitted by

MEMBER 1: _____ STUDENT ID: _____

MEMBER 2: _____ STUDENT ID: _____

MEMBER 3: _____ STUDENT ID: _____

of

BACHELOR OF ENGINEERING / TECHNOLOGY

in

DEPARTMENT: _____

INSTITUTION: _____

A Java desktop workspace for focused email and explainable spam review

November 2026

BONAFIDE CERTIFICATE

Certified that this project report titled “MAILDESK — EMAIL MANAGEMENT SYSTEM” is the bonafide work of the students listed below, who carried out the project as part of the integrated Object-Oriented Programming and Database Management Systems course during the academic year 2026–2027.

The project implements a Java Swing email workspace that connects to Gmail through Google OAuth, stores an account-scoped message sample in MySQL, and provides transparent rule-based spam review with user correction. The attached report describes the current implementation and distinguishes implemented features from proposed work.

STUDENT SIGNATURES

STUDENT	NAME / ID	SIGNATURE
Member 1	_____	_____
Member 2	_____	_____
Member 3	_____	_____

PROJECT GUIDE / COURSE INSTRUCTOR

PROJECT GUIDE	COURSE INSTRUCTOR
Signature: _____	Signature: _____
Name: _____	Name: _____
Department: _____	Department: _____

INTERNAL EXAMINER: _____ EXTERNAL EXAMINER: _____

ABSTRACT

Maildesk is a Java desktop email-management system that brings common Gmail tasks into one focused workspace. A user can connect through Google OAuth, browse Inbox, Sent, and Gmail Spam, search the locally cached sender, recipient, subject, and body, read messages, and compose plain-text email. The Swing application uses an application service boundary so the interface, Gmail integration, local spam analysis, and persistence can be understood as separate parts.

The project's distinctive feature is an explainable second opinion for suspicious messages. A deterministic Java rule classifier assigns a score from visible signals such as blocked senders, suspicious phrases, many links, and urgent subject wording. The interface presents reasons with the score so the mailbox owner can review and correct the decision. A score is a rule score, not a probability. Gmail's own Spam label remains distinct from Maildesk's locally suspected messages. A saved user correction is retained per account.

For predictable demonstrations, the live connection and refresh retrieve up to six recent messages from each of Inbox, Sent, and Gmail Spam, deduplicate message IDs, and cap the local sample at 18 unique messages. The application stores accounts, messages, recipients, labels, spam results, feedback, and blocked senders in MySQL through JDBC. Composite account/message keys scope cached records to an account. OAuth tokens and database passwords remain in local configuration and are not stored in the message tables.

The separate demo launcher uses fictional in-memory messages. It does not connect to Gmail or MySQL, use the network, or deliver real email. The project uses Java 17 or newer, Swing, Maven, the Gmail REST API, Google OAuth, and MySQL 8. The report documents its flow, use cases, data model, modules, source-level implementation, screenshots, setup, and verification coverage. Attachments, rich text, scheduled synchronization, broad public OAuth distribution, and machine-learning classification are future work, not current features.

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE NO.
1	INTRODUCTION: problem statement, project overview, purpose, scope	5–8
2	FUNCTIONALITIES PROVIDED	9
3	SYSTEM DESIGN: system flow, dataflow, use cases, ER model, architecture	10–14
4	MODULE DESCRIPTION: workspace, authorization, mailbox, messages, spam, demo	15–24
5	IMPLEMENTATION: Java contracts, Gmail, spam rules, JDBC, setup, verification	25–41
6	CONCLUSION	42
7	REFERENCES	43

Names, student IDs, department, institution, project guide, and instructor details are deliberately left as fill-in fields. The supplied example's personal and institutional information has not been copied into this report.

1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Email users often move between separate mailbox views to find recent work, inspect a message, compose a reply, and check suspicious requests. A basic spam filter can hide a message or provide little explanation for its decision. When users cannot see the reason, they have less confidence in the result and may have difficulty correcting a false positive.

A student-scale email management application should combine the common tasks in a clear interface while keeping the user involved in security decisions. It also needs a reliable way to preserve a small mailbox cache in a relational database without turning the project into a full Gmail mirror.

Maildesk addresses this problem with a focused desktop workspace, a bounded recent-message sample, MySQL persistence, and an explainable local spam review. The user sees matched warning reasons and can mark a received message as spam or not spam. Gmail's own Spam folder and Maildesk's local suggestions remain separate views.

1 - INTRODUCTION

1.2 PROJECT OVERVIEW

Maildesk is a Java Swing desktop client connected to Gmail through Google's OAuth flow and Gmail REST API. Its main window presents a navigation rail, mailbox list, search controls, a message reader pane, and a compose form. JDBC connects its application cache to a local MySQL database.

The core service is exposed through an EmailService interface. GmailEmailService coordinates remote Gmail actions with the local store; DemoEmailService implements the same interface using fictional in-memory messages. Because both implementations share a contract, the interface can be demonstrated without credentials, network access, or a running database.

On live startup, the system requests six recent items from Inbox, Sent, and Spam, deduplicates IDs, then stores no more than 18 unique messages. Local spam analysis attaches a score and reasons to received messages. Search covers the cached sample. Compose sends plain-text email after validation. The database persists message data, Gmail labels, spam explanations, feedback, and blocked senders per account.

1 - INTRODUCTION

1.3 PURPOSE AND OBJECTIVES

The purpose of Maildesk is to demonstrate an end-to-end Java and DBMS application that makes email work easier to understand. The system centralizes reading, searching, writing, and review while showing how an object-oriented service layer can connect a Swing interface to both an external API and a relational database.

Project objectives

- Connect a Gmail account by using Google OAuth in the system browser; do not collect the mailbox password.
- Show Inbox, Sent, Gmail Spam, and locally Suspected Spam in a consistent desktop interface.
- Search the cached sender, recipient, subject, and plain-text body.
- Validate a plain-text draft before requesting a Gmail send.
- Persist account-scoped messages, recipients, Gmail labels, spam reasons, and user corrections through JDBC.
- Explain why the local rule classifier suggested review and allow the user to correct it.
- Provide a fictional offline demonstration that is safe to show without connecting to a real account.

1 - INTRODUCTION

1.4 SCOPE

CURRENT PROJECT SCOPE	OUTSIDE THE CURRENT BUILD
Gmail Desktop OAuth; Inbox, Sent, Gmail Spam; recent-message cache; local search.	Hosted multi-user mail service or a complete replacement for Gmail.
Read, send plain-text, refresh, mark spam / not spam.	Attachments, rich-text editing, and scheduled synchronization.
MySQL persistence via JDBC and one active connected account.	Enterprise administration or public multi-account hosting.
Transparent deterministic spam rules and account-specific feedback.	Machine-learning classification or a probability-calibrated risk score.
Offline demo with fictional, in-memory mail.	Real delivery from demo mode or database persistence in demo mode.

The live application uses Google permissions configured for the Gmail API. Distribution to a broad public audience may require the applicable Google consent-screen and OAuth verification work. The Windows package contains the application runtime, but each live installation still requires local MySQL and a user-provided Desktop OAuth client.

2 - FUNCTIONALITIES PROVIDED

FUNCTIONALITIES PROVIDED

Maildesk offers a compact set of workflows that can be explained and demonstrated from one workspace. A live mailbox owner signs in through Google, then chooses one of the four mailbox views. The interface updates the list and reader pane without replacing the connected account.

FUNCTION	USER VALUE	BOUNDARY
Connect Gmail	Use Google sign-in and load recent mail.	Requires configured Desktop OAuth credentials and consent.
Inbox / Sent / Gmail Spam	Read common mailbox categories.	Gmail labels identify provider folders.
Suspected Spam	Review the local rule model’s suggestions.	Separate from Gmail’s Spam folder.
Search	Find a message in the cached sample.	Does not search every remote Gmail message.
Compose	Send a validated plain-text message.	No attachments or rich formatting.
Correct spam	Save the user’s decision and update Gmail.	Only explicit user actions change Gmail labels.
Offline demo	Explore fictional data without external setup.	No network, MySQL, or real mail delivery.

Refresh replaces the local recent snapshot; it does not delete Gmail messages. Failed reads do not become an empty successful result. A local correction is associated with the account and may override the later automatic rule result.

3 - SYSTEM DESIGN

3.1 SYSTEM FLOW DIAGRAM

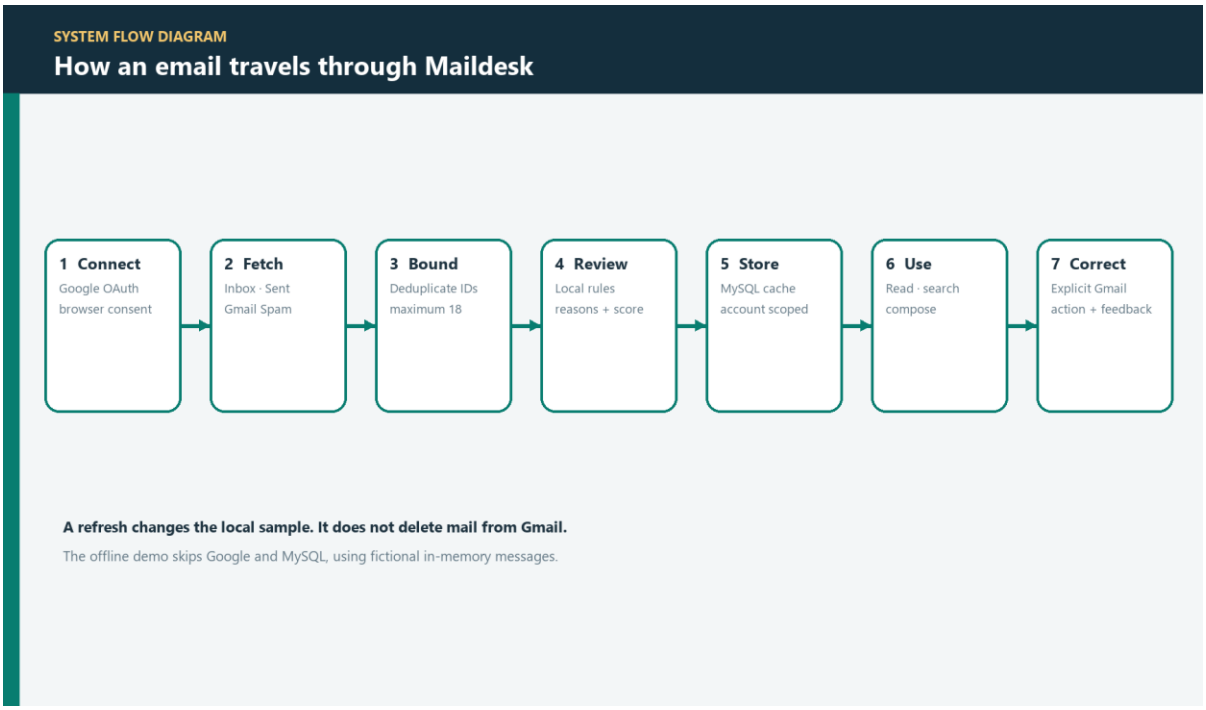


Figure 1: System flow from account connection through user review

The flow starts when a user connects Gmail. Maildesk requests up to six recent messages from each supported provider label. IDs returned from more than one label are deduplicated before the 18-item cap is applied. The service evaluates received messages locally and writes the snapshot and analysis into MySQL.

The person then reads, searches, or composes. Gmail state changes occur only after a corresponding user action. A spam correction is also saved to local feedback. In offline demo mode, the connection step is simulated and the data path remains in memory.

3 - SYSTEM DESIGN

3.2 DATAFLOW DIAGRAM

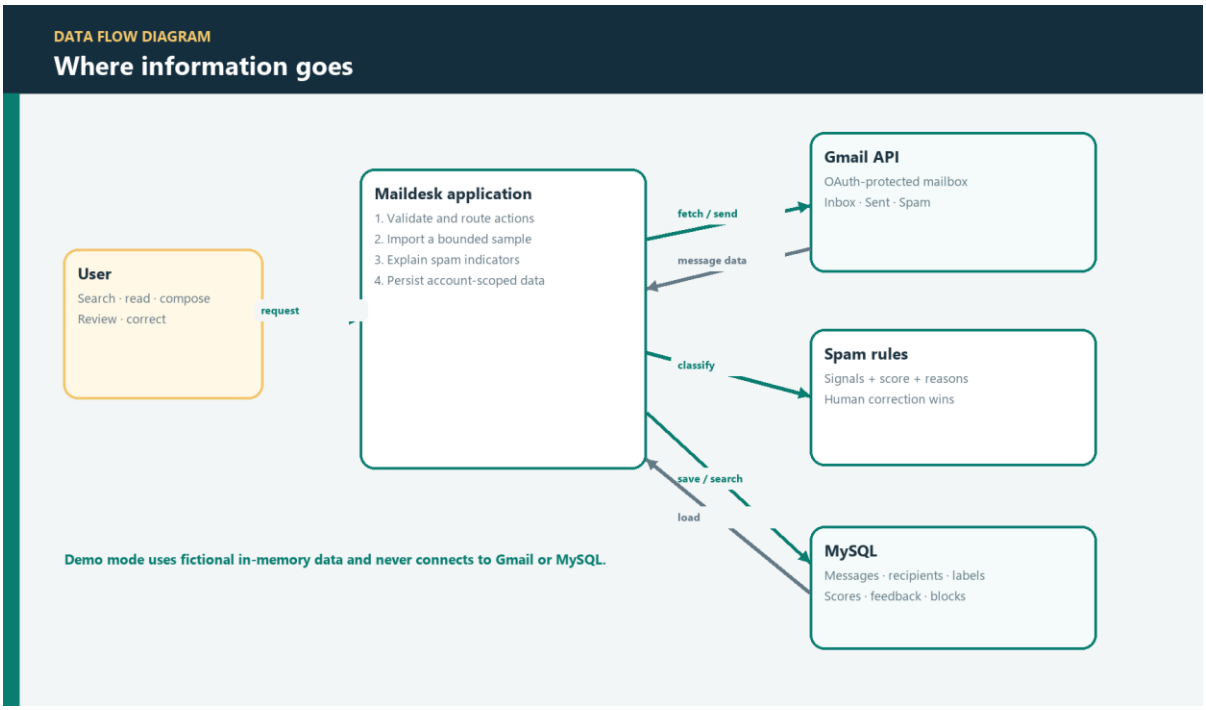


Figure 2: Data flow between the person, application, Gmail, classifier, and MySQL

The Swing interface accepts searches and explicit actions. The application service validates the selected mailbox or draft and routes requests to the Gmail gateway or the database store. Gmail returns message metadata and body text; the local classifier derives a score and explanation from the message fields and account-specific blocked-sender list.

The database stores a cache and the results needed for the local UI. OAuth tokens remain in a local protected token store rather than in the email tables. The demo implementation bypasses all external boxes and reads fictional fixtures from memory.

3 - SYSTEM DESIGN

3.3 USE CASE DIAGRAM

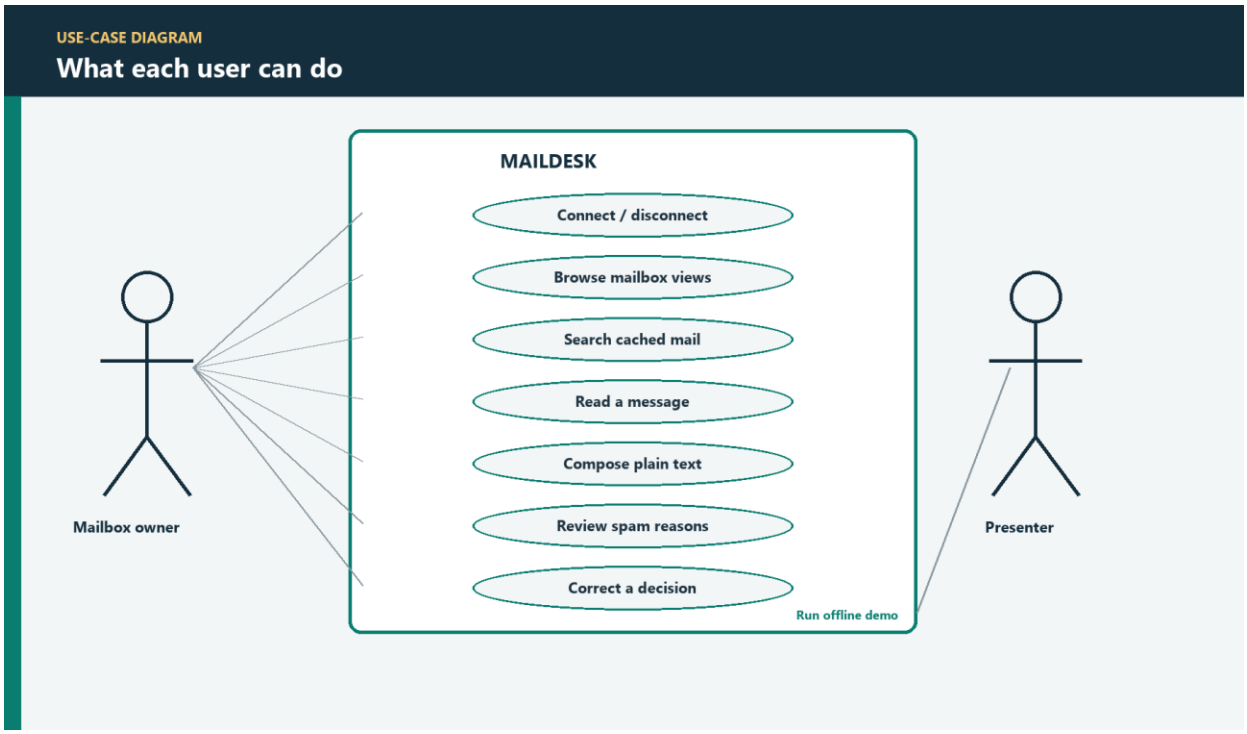


Figure 3: Mailbox owner use cases and the presenter's demo entry

ACTOR	USE CASE	DESCRIPTION
Mailbox owner	Connect / disconnect	Authorize a Gmail account or end the local session.
Mailbox owner	Browse / search	Choose Inbox, Sent, Spam, or Suspected Spam; search the cache.
Mailbox owner	Read / compose	Open a cached message or compose a validated plain-text message.
Mailbox owner	Review / correct	Read rule reasons and mark a received message spam or not spam.
Presenter	Run demo	Explore fictional content without Gmail or database setup.

3 - SYSTEM DESIGN

3.4 ENTITY RELATIONSHIP DIAGRAM

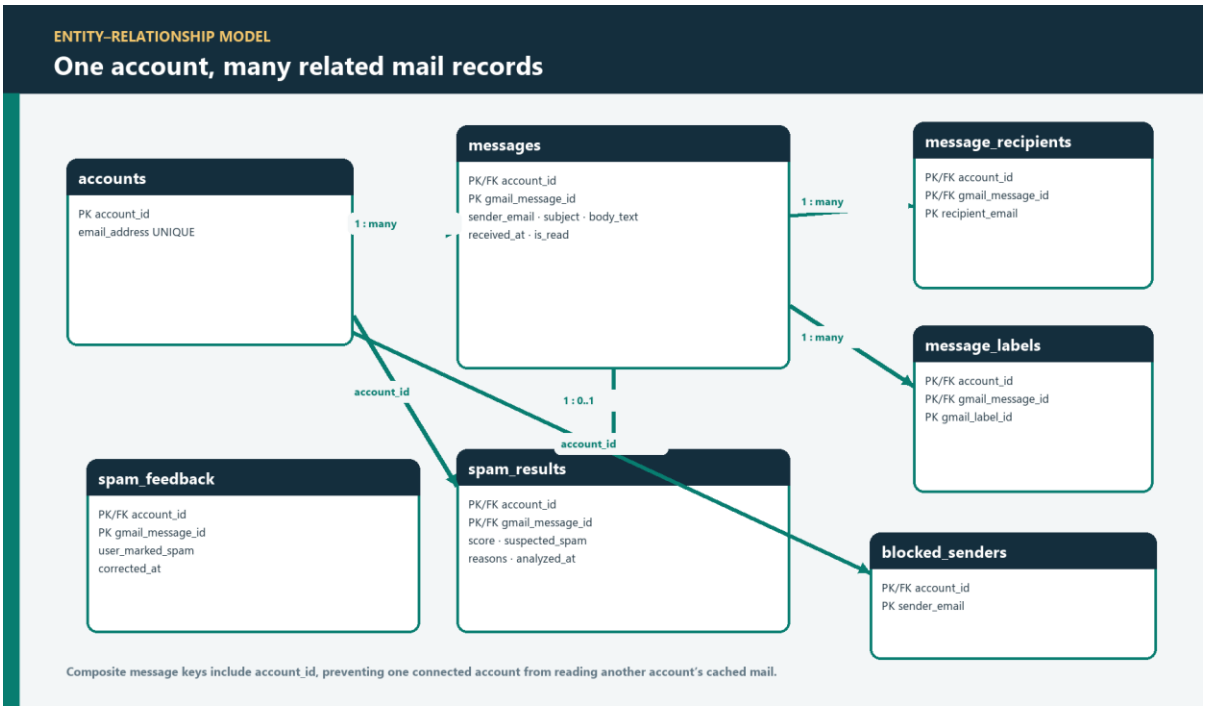


Figure 4: MySQL entities and account-scoped key relationships

The accounts table identifies each connected mailbox. Messages use the composite key (account_id, gmail_message_id). Recipient and label rows reference that key. Spam results are stored per message, while feedback and blocked-sender rows are associated with the account. This structure keeps repeated recipients and Gmail labels out of a single overloaded message column.

Foreign keys enforce parent-child relationships. The composite message key ensures lookups and writes are scoped to the active account. sql/schema.sql contains the full table definitions and constraints.

3 - SYSTEM DESIGN

3.5 SOFTWARE ARCHITECTURE

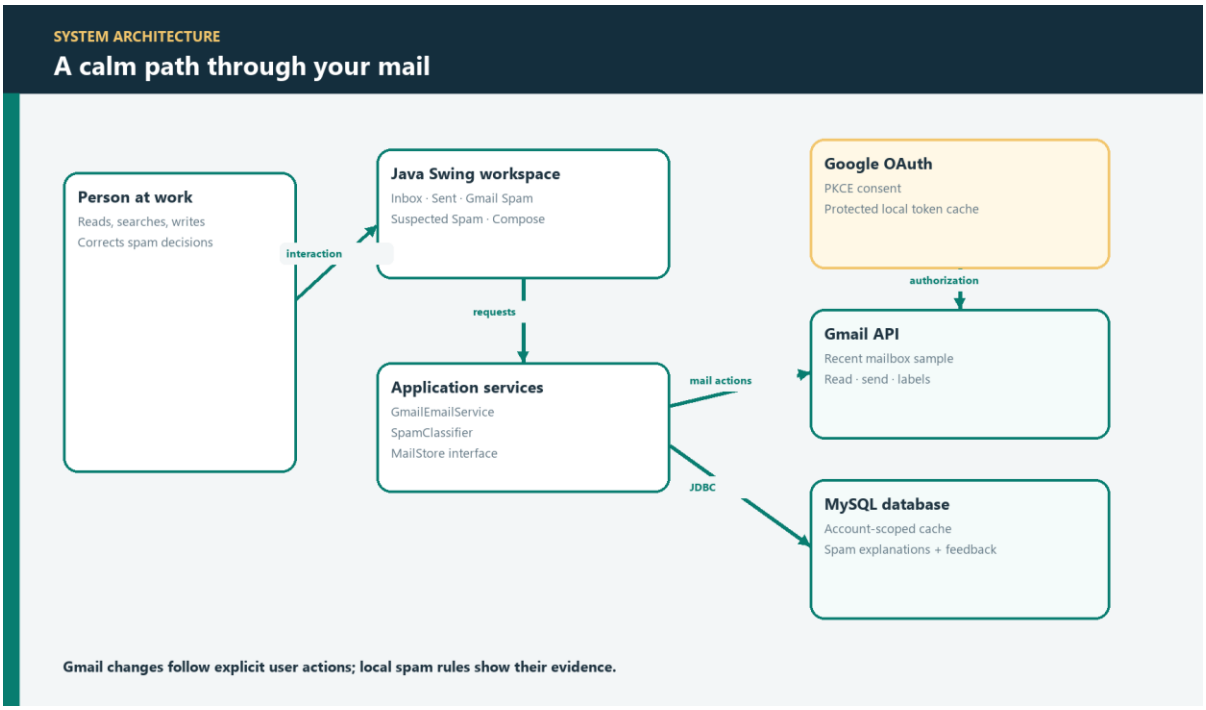


Figure 5: Swing, service, Gmail, OAuth, spam analysis, and persistence layers

The Swing UI depends on EmailService rather than on GmailApiClient or JdbcMailStore directly. GmailEmailService is the live implementation. DemoEmailService uses the same application boundary with fictional data. This separation allows interface behavior to be reviewed without granting real Gmail permissions.

The Gmail package owns OAuth and REST details. The storage package owns the MailStore contract and its JDBC implementation. SpamClassifier is a deterministic local component. Model classes carry immutable message and result values between these layers.

4 - MODULE DESCRIPTION

MODULE OVERVIEW

The project is divided into related modules so each function has a clear role. The interface module handles presentation and user actions. The service module coordinates business operations. Gmail, spam, and storage modules implement integrations behind stable boundaries.

MODULE	MAIN RESPONSIBILITY	KEY CLASSES
Welcome / account	Explain product and start OAuth.	MailAppPanel, GmailAuth
Mailbox workspace	Navigation, table, reader, search.	MainFrame, MailAppPanel, MailTableModel
Compose	Validate and submit plain-text draft.	ComposePanel
Gmail integration	Authorize, fetch, read, send, label.	GmailApiClient, GmailEmailService
Spam review	Calculate score and show reasons.	SpamClassifier, SpamResult
Database	Persist messages and feedback.	MailStore, JdbcMailStore
Offline demo	Simulate screen actions with fictional data.	DemoApp, DemoEmailService

The following pages explain each visible module and connect it to the implementation that supports it.

4 - MODULE DESCRIPTION

4.1 WELCOME AND CONNECT SCREEN

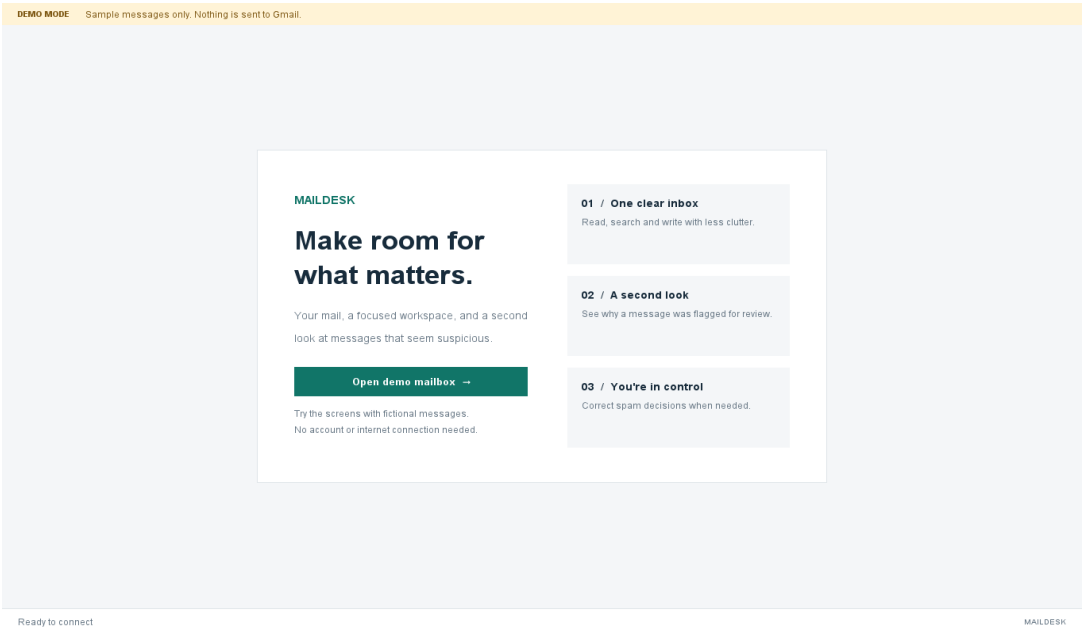


Figure 6: Maildesk welcome screen in the offline demonstration

The welcome page introduces the three product ideas: a clear inbox, a second look at suspicious messages, and user control over the decision. The production button begins Gmail authorization. The demo button opens the same mailbox workspace with fictional content.

The connected-account label and status message communicate whether Maildesk is ready. The page does not ask the user to type a Gmail password. In demo mode a banner explicitly states that sample messages are shown and nothing is sent to Gmail.

4 - MODULE DESCRIPTION

4.2 GOOGLE AUTHORIZATION MODULE

Gmail access uses an OAuth Desktop client and the system browser. GmailAuth generates a random state and PKCE verifier, derives an S256 challenge, starts an HTTP callback bound to 127.0.0.1, and opens Google's authorization page. After Google returns a code to the local callback, the app exchanges the code and verifier for tokens.

A saved local token may be refreshed without collecting the user's password. The token store protects the local files and the repository ignores credentials and token directories. The Gmail scope used by the application is gmail.modify because reading messages and updating labels require those operations.

A Google project in testing mode must include the user as a test user. The app's local sign-in does not bypass Google consent settings or public-app verification requirements.

4 - MODULE DESCRIPTION

4.3 MAILBOX NAVIGATION

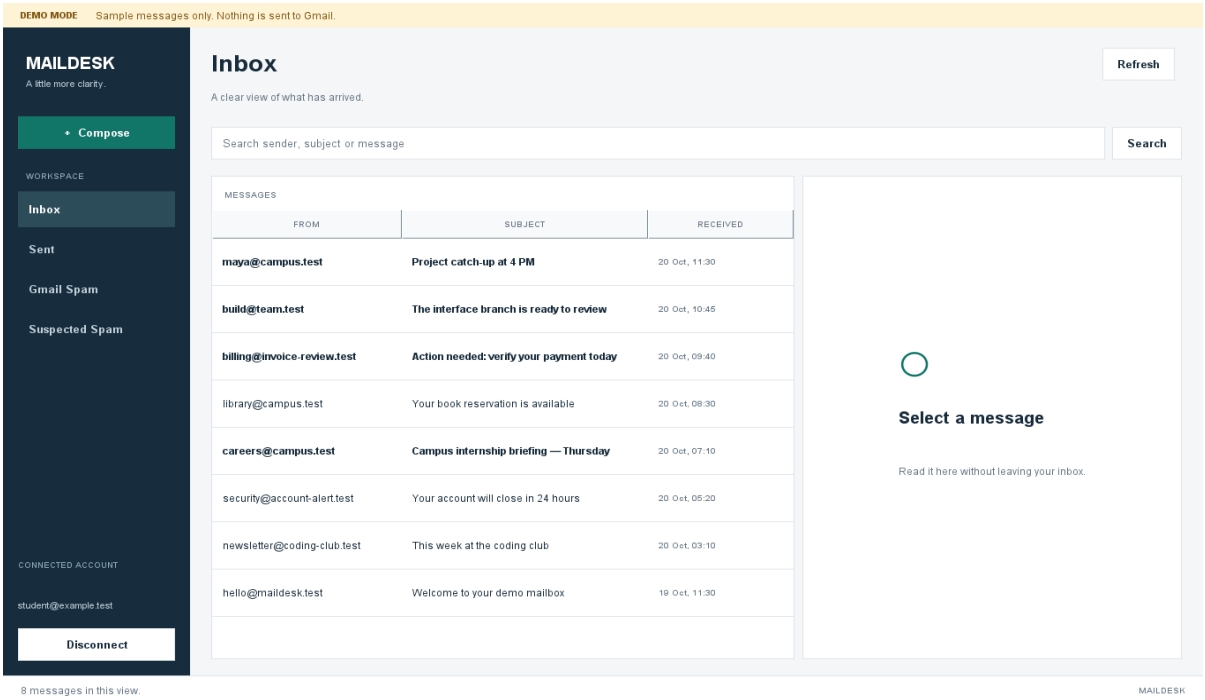


Figure 7: Inbox navigation, message table, search controls, and reader pane

The left navigation presents Inbox, Sent, Gmail Spam, and Suspected Spam. Selecting a mailbox changes the view title and retrieves messages through the shared service. The central table shows sender or recipient, subject, and time. The right pane is reserved for message contents and the spam review.

The table model presents an immutable view of the selected results. A single selected row is opened in the reader pane. Empty search results and empty folders have dedicated states so an empty list is not confused with a broken connection.

4 - MODULE DESCRIPTION

4.4 RECENT SAMPLE AND REFRESH

The live client intentionally works with a bounded sample rather than downloading the entire mailbox during first connection. GmailEmailService requests six recent messages for each of Inbox, Sent, and Spam. The Gmail API client deduplicates message IDs, and the service applies an overall limit of 18.

```
List<EmailMessage> snapshot = gmail.fetchRecentMessages(accountId, 6)
        .stream().limit(18).toList();
verifyAccount(account, snapshot);
store.replaceSnapshot(accountId, snapshot);
```

Refresh repeats the same bounded workflow. Replacing the local snapshot removes records from the local cache if they are absent from the latest sample, but it does not delete messages from Gmail. A refresh failure is reported as an error and the previous cache is preserved.

This boundary is useful for a small class demonstration: the count is easy to inspect, and the behavior does not imply a complete synchronization engine.

4 - MODULE DESCRIPTION

4.5 SEARCH AND MESSAGE READER

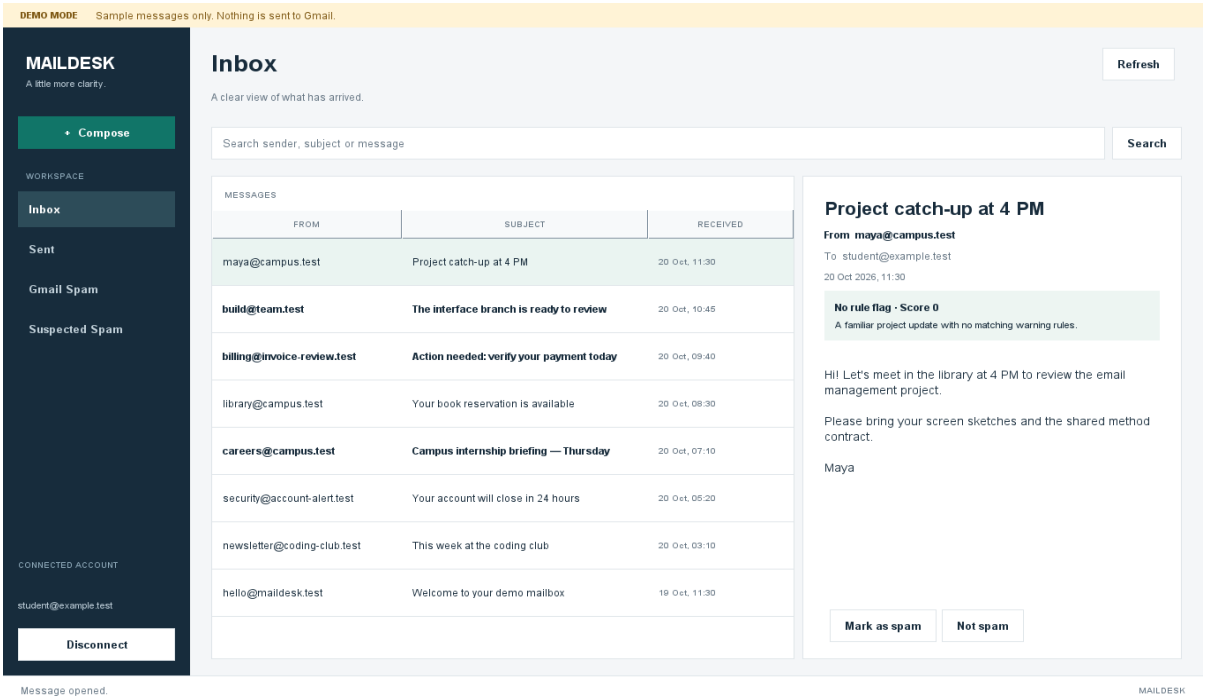


Figure 8: A message is read beside the list and its current local review result

Search matches sender, recipients, subject, and body text in the local sample. It does not claim to search every message stored in Gmail. Results are sorted newest first. Selecting a message asks the service to load the latest body and, when necessary, update Gmail's unread status.

The reader pane displays the sender, recipient, subject, date, body, and spam reason card. Plain text is rendered as plain text so message content cannot be interpreted as Swing HTML. Long bodies can scroll vertically, keeping the end of a message reachable at common window widths.

4 - MODULE DESCRIPTION

4.6 COMPOSE MODULE



Figure 9: Plain-text compose form with a visible offline-demo notice

The compose form collects one recipient address, a subject, and a plain-text body. Before handing the draft to the service, the form checks that the recipient is valid, the subject contains 1–200 characters on one line, and the body is non-empty and within the configured limit.

The production service validates again before sending. This protects the application boundary even if a different interface implementation is introduced later. The demo compose form keeps the same validation behavior but stores the result only in its local Sent view.

Attachments, rich-text formatting, draft synchronization, and multiple recipients are not part of this version.

4 - MODULE DESCRIPTION

4.7 SUSPECTED SPAM MODULE

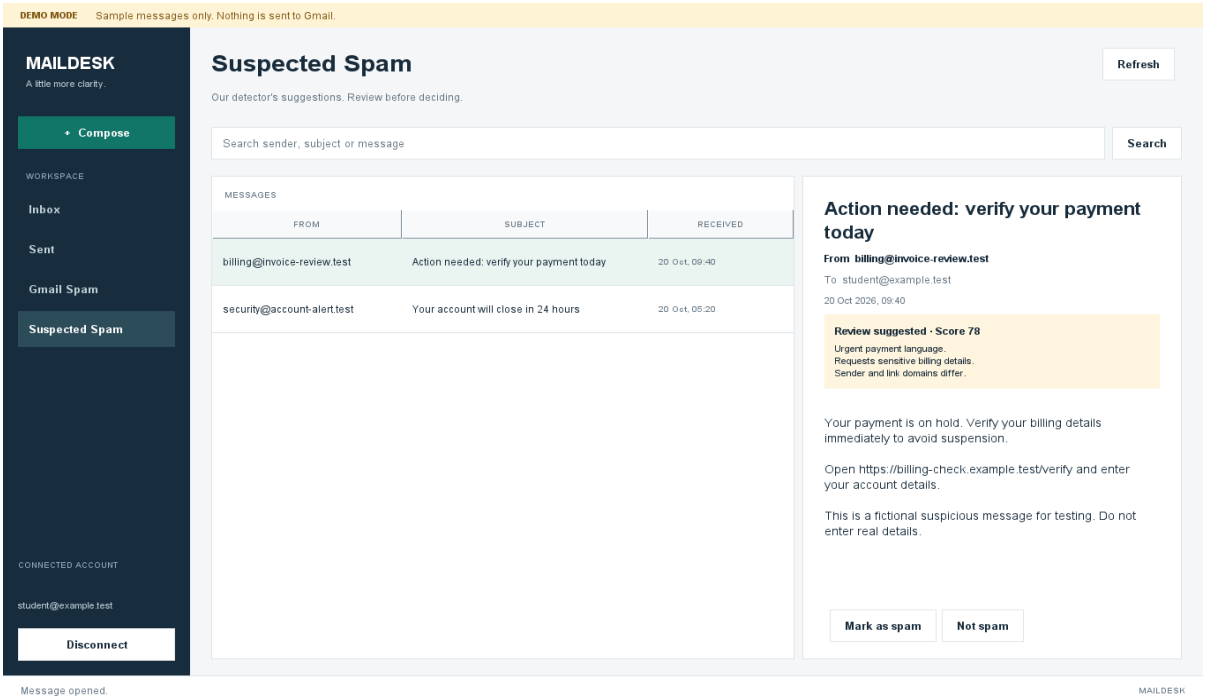


Figure 10: The local review view shows evidence beside a fictional suspicious message

Suspected Spam is a local review queue generated from Maildesk's rules. It does not copy or rename Gmail's Spam label. The message detail shows the rule score and each reason that contributed to it. The user can choose Mark as spam or Not spam after reviewing the evidence.

The demonstration screenshot uses a fictional invoice message and a reserved .test domain. Its threatening wording, link mismatch, and sensitive-data request are written as sample signals for the interface demonstration; they are not evidence from a real account.

4 - MODULE DESCRIPTION

4.8 USER CORRECTION AND LABEL ACTIONS

A user correction has two effects in the live application: Maildesk asks Gmail to apply or remove the Spam label, and the local database saves the user's correction. A correction is associated with the connected account and message ID. On a later analysis, stored feedback takes precedence over the automatic classifier result.

USER ACTION	GMAIL EFFECT	LOCAL EFFECT
Mark as spam	Add SPAM and remove INBOX where applicable.	Save spam feedback and current labels.
Not spam	Remove SPAM and add INBOX where applicable.	Save not-spam feedback and current labels.
Open unread message	Remove UNREAD through Gmail.	Save latest message and read state.
Disconnect	End local authorization session.	Clear active account from the UI; does not delete Gmail or DB rows.

The application does not automatically replay a failed send or label change. This avoids accidental duplicate mail or repeated changes after a network error.

4 - MODULE DESCRIPTION

4.9 STANDALONE DEMONSTRATION MODULE

The standalone demo is an offline presentation mode built against the same EmailService interface as the live client. DemoEmailService seeds fictional messages in memory, supports mailbox navigation, search, message reading, local demo send, and spam corrections.

DEMO ACTION	RESULT
Open demo mailbox	Shows fictional Inbox, Sent, Gmail Spam, and Suspected Spam.
Search	Filters the in-memory sample.
Compose / send	Creates a demo Sent item; no email is delivered.
Mark spam / not spam	Changes only local demo labels and correction state.
Restart application	Reloads the seed sample; demo changes are not persistent.

The banner is part of the safety and presentation design. It states that sample messages are used and no email is sent through Gmail. No account or internet connection is required to demonstrate the interface.

5 - IMPLEMENTATION

5.1 SOURCE ORGANIZATION

SOURCE AREA	RESPONSIBILITY
src/main/java/ems/ui	Swing frames, panels, table model, compose form, visual theme.
src/main/java/ems/service	EmailService contract and live Gmail service coordination.
src/main/java/ems/gmail	Google OAuth, token persistence, Gmail REST and MIME message helpers.
src/main/java/ems/storage	MailStore boundary, JDBC store, database schema setup.
src/main/java/ems/model	Account, mailbox, message, and spam-result domain objects.
src/main/java/com/emailapp/spam	Deterministic rule classifier and score weights.
src/demo/java	Offline demo app and fictional in-memory service.
src/test/java and sql/	Verification code and MySQL schema.

The shared application boundary avoids coupling the interface directly to JDBC or HTTP code. The project uses Maven and the Maven Wrapper to compile Java 17 source and produce an executable JAR. A Windows packaging script uses jpackage to create an app image with a bundled runtime.

5 - IMPLEMENTATION

5.2 SHARED SERVICE CONTRACT

The EmailService interface is the method-name contract shared by the Swing UI, the live Gmail integration, and the offline demo. The UI depends on these operations and does not need to know which implementation is active.

```
Account connectGmail() throws AppException;
void disconnect();
void refresh() throws AppException;
List<EmailMessage> getMessages(Mailbox mailbox, String query) throws AppException;
EmailMessage openMessage(String gmailMessageId) throws AppException;
String sendMessage(String recipientEmail, String subject, String body) throws
AppException;
SpamResult getSpamResult(String gmailMessageId) throws AppException;
void markAsSpam(String gmailMessageId) throws AppException;
void markAsNotSpam(String gmailMessageId) throws AppException;
```

AppException carries a recoverable message suitable for a user notice. Database and provider details are translated into bounded, non-sensitive descriptions. The contract calls that can wait for network or storage work execute on a background worker rather than the Swing event thread.

5 - IMPLEMENTATION

5.3 DOMAIN MODEL AND OBJECT-ORIENTED DESIGN

EmailMessage is an immutable snapshot. It holds an account ID, Gmail message ID, sender, recipient list, subject, plain-text body, received time, and provider label IDs. Lists and sets are copied when the model is created, so callers cannot mutate the stored collections through an earlier reference.

```
public final class EmailMessage {
  private final long accountId;
  private final String gmailMessageId;
  private final String senderEmail;
  private final List<String> recipientEmails;
  private final String subject;
  private final String bodyText;
  private final Instant receivedAt;
  private final Set<String> gmailLabelIds;
  // Constructor copies recipientEmails and gmailLabelIds.
}
```

Account contains a local database identifier and an email address; OAuth credentials do not belong in this object. Mailbox is an enum with INBOX, SENT, SPAM, and SUSPECTED. SpamResult contains a rule score, suspected flag, and immutable explanation list.

5 - IMPLEMENTATION

5.4 SWING UI AND BACKGROUND WORKERS

Swing components must be changed on the event-dispatch thread. Network requests and JDBC operations can take time, so MailAppPanel uses SwingWorker. The busy flag prevents duplicate user actions while the operation is in flight. On completion, the UI displays the result or a safe error message.

```
new SwingWorker<T, Void>() {
  @Override protected T doInBackground() throws Exception {
    return work.call();
  }
  @Override protected void done() {
    busy = false;
    try {
      T result = get();
      success.accept(result);
    } catch (ExecutionException error) {
      failure.accept(userMessage(error.getCause()));
    } finally { updateControls(); }
  }
}.execute();
```

The reader and compose forms use reusable Swing panels. MailTableModel presents the selected mailbox rows. UI checks exercise these screens headlessly through the same service boundary used by the live application.

5 - IMPLEMENTATION

5.5 GOOGLE OAUTH AUTHORIZATION CODE FLOW

GmailAuth implements Authorization Code with PKCE for a native desktop client. The app creates a random state and verifier, hashes the verifier with SHA-256, and binds a temporary callback listener to the IPv4 loopback address. The browser opens Google's authorization endpoint with the challenge and requested Gmail scope.

```
String state = randomToken();
String verifier = randomToken();
String challenge = Base64.getUrlEncoder().withoutPadding()
    .encodeToString(MessageDigest.getInstance("SHA-256")
    .digest(verifier.getBytes(StandardCharsets.US_ASCII)));
server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
URI redirect = URI.create("http://127.0.0.1:"
    + server.getAddress().getPort() + "/oauth2callback");
form.put("state", state);
form.put("code_challenge", challenge);
form.put("code_challenge_method", "S256");
```

The callback validates state before accepting the authorization code. Maildesk then exchanges the code and verifier for tokens. A bounded timeout produces a clear sign-in error. The local callback server is always stopped in a finally block.

5 - IMPLEMENTATION

5.6 GMAIL REST AND MESSAGE OPERATIONS

GmailApiClient is the network adapter behind GmailGateway. It obtains an access token from the token provider and makes HTTP requests to the Gmail API. The service obtains the account profile after authorization, then retrieves message metadata and content through message IDs.

OPERATION	APPLICATION BEHAVIOR
Profile	Read the authorized account email address.
List messages	Request messages with the chosen Gmail labels and query.
Read message	Load sender, recipient, subject, plain-text body, timestamp, and labels.
Send	Build and submit a MIME message with one recipient and plain-text body.
Modify labels	Update unread or spam status only after an explicit user action.

The application never sends mail as part of automated tests. The user sees status text when an operation fails, and the UI does not mistake a local cache error for a successful remote operation.

5 - IMPLEMENTATION

5.7 BOUNDED RECENT-MAIL IMPORT

The recent-message path queries three Gmail labels independently: INBOX, SENT, and SPAM. Each request is limited to a per-label count. A linked set deduplicates IDs that occur in multiple categories. The service applies an overall limit before replacing the local snapshot.

```
for (String label : List.of("INBOX", "SENT", "SPAM")) {
  JsonObject page = request("GET", recentPath(label, limit), null);
  for (var item : page.getAsJsonArray("messages")) {
    ids.add(requiredString(item.getAsJsonObject(), "id"));
  }
}
return readMessages(accountId, ids);
```

GmailEmailService calls fetchRecentMessages(accountId, 6) and limits the combined sample to 18. The schema uses the Gmail message ID as part of its primary key, so repeated refreshes update the same cached record rather than creating duplicates.

The phrase “sample” is deliberate: the application is not presented as a complete Gmail mirror or as a full synchronization engine.

5 - IMPLEMENTATION

5.8 EXPLAINABLE SPAM CLASSIFIER

SpamClassifier is a deterministic rule model. It begins at zero, adds points for matched evidence, and stores a reason for each contributing rule. A blocked sender adds 50 points; each matched suspicious phrase adds 20 up to three matches; five or more links add 15; and urgent subject wording adds 10. The review threshold is 50.

```
if (blockedSender) {
  score += SpamRules.BLOCKED_SENDER;
  reasons.add("Sender is blocked");
}
if (content.contains(phrase)) {
  score += SpamRules.SUSPICIOUS_PHRASE;
  reasons.add("Suspicious phrase: " + phrase);
}
if (linkCount >= 5) {
  score += SpamRules.MANY_LINKS;
  reasons.add("Message contains many links");
}
if (score >= SpamRules.SPAM_THRESHOLD) {
  suspectedSpam = true;
}
```

The score is not machine learning and is not a probability. The purpose of this module is an inspectable second opinion. Feedback stored by JdbcMailStore can override the later automatic suggestion for that account and message.

5 - IMPLEMENTATION

5.9 DATABASE SCHEMA: ACCOUNTS AND MESSAGES

The accounts table has a generated account_id and a unique email_address. The messages table is keyed by account_id and gmail_message_id. Its columns store sender, subject, body text, received time, and read state.

```
CREATE TABLE accounts (
  account_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  email_address VARCHAR(254) NOT NULL UNIQUE
);

CREATE TABLE messages (
  account_id BIGINT NOT NULL,
  gmail_message_id VARCHAR(128) NOT NULL,
  sender_email VARCHAR(320) NOT NULL,
  subject TEXT NOT NULL,
  body_text LONGTEXT NOT NULL,
  received_at DATETIME(6) NOT NULL,
  is_read BOOLEAN NOT NULL DEFAULT TRUE,
  PRIMARY KEY (account_id, gmail_message_id),
  FOREIGN KEY (account_id) REFERENCES accounts(account_id)
);
```

The database uses utf8mb4. The project schema creates email_management when permitted and initializes its tables from sql/schema.sql. A dedicated database user should be used for the local application.

5 - IMPLEMENTATION

5.10 NORMALIZED RECIPIENTS AND LABELS

Recipients and Gmail labels are stored as child records. This allows a message to have multiple recipients and labels without placing a delimiter-encoded list in a single database cell. The child rows reference the same composite key as the parent message.

```
CREATE TABLE message_recipients (
  account_id BIGINT NOT NULL,
  gmail_message_id VARCHAR(128) NOT NULL,
  recipient_email VARCHAR(320) NOT NULL,
  PRIMARY KEY (account_id, gmail_message_id, recipient_email),
  FOREIGN KEY (account_id, gmail_message_id)
    REFERENCES messages(account_id, gmail_message_id) ON DELETE CASCADE
);

CREATE TABLE message_labels (
  account_id BIGINT NOT NULL,
  gmail_message_id VARCHAR(128) NOT NULL,
  gmail_label_id VARCHAR(128) NOT NULL,
  PRIMARY KEY (account_id, gmail_message_id, gmail_label_id),
  FOREIGN KEY (account_id, gmail_message_id)
    REFERENCES messages(account_id, gmail_message_id) ON DELETE CASCADE
);
```

The composite foreign key ensures a label or recipient cannot be attached to a message in another account. Cascading deletion removes these child rows when a locally cached parent message is removed.

5 - IMPLEMENTATION

5.11 SPAM RESULTS, FEEDBACK, AND BLOCKED SENDERS

TABLE	IMPORTANT FIELDS	ROLE
spam_results	account_id, gmail_message_id, score, suspected_spam, reasons, analyzed_at	Stores the latest automatic explanation.
spam_feedback	account_id, gmail_message_id, user_marked_spam, corrected_at	Stores the user's saved correction.
blocked_senders	account_id, sender_email	Adds an account-specific classifier signal.

Spam results reference the message composite key. Feedback is scoped to the account and intentionally can survive replacement of the small message sample. Blocked senders use the account ID and normalized address as their key.

When the store analyzes a message, it loads the account's blocked senders and passes the matching state into SpamClassifier. The user's explicit feedback remains a separate record rather than being folded into the automatic score.

5 - IMPLEMENTATION

5.12 JDBC PERSISTENCE AND TRANSACTIONS

JdbcMailStore implements the MailStore contract with prepared statements. Snapshot replacement and related record writes run in a transaction. The store upserts the message, replaces recipient and label rows, recalculates automatic spam reasons, and removes cached messages that are no longer in the current bounded snapshot.

```
private void transaction(SqlWork work) throws SQLException {
  try (Connection connection = connections.open()) {
    connection.setAutoCommit(false);
    try {
      work.run(connection);
      connection.commit();
    } catch (SQLException error) {
      connection.rollback();
      throw error;
    }
  }
}
```

The transaction groups the local cache update so a partial snapshot is not treated as complete. The code checks that every message belongs to the account ID passed to the store. SQL values are bound through PreparedStatement parameters rather than concatenated into user-facing query strings.

5 - IMPLEMENTATION

5.13 CONFIGURATION AND DATA PROTECTION

AppConfig reads local settings and supports environment-variable overrides. The documented properties include the database URL, username, password, Gmail Desktop credentials path, and local token directory. Sensitive local config files are ignored by Git.

SETTING	EXAMPLE PROPERTY / VARIABLE	PURPOSE
Database URL	db.url / DB_URL	Select local MySQL database.
Database login	db.user, db.password / DB_USER, DB_PASSWORD	Authenticate the application to MySQL.
OAuth JSON	gmail.credentials / GMAIL_CREDENTIALS	Point to the Google Desktop OAuth file.
Token directory	gmail.tokenDir / GMAIL_TOKEN_DIR	Keep saved sign-in tokens outside source control.

The repository excludes OAuth credentials, token files, passwords, and private mail exports. Do not put these values in a report, source file, screenshot, issue, or shared GitHub commit. Use a dedicated local MySQL account with permission to access only the project database.

5 - IMPLEMENTATION

5.14 ERROR HANDLING AND RESPONSIVENESS

The application surfaces errors in terms the user can act on. A missing credential path points to setup instructions. A database error points to db.url, db.user, and db.password. A provider rate limit tells the user to retry after a temporary delay. Sensitive provider payloads and password values are not included in ordinary status messages.

SwingWorker runs service operations away from the UI event thread. Controls are disabled while an operation is in flight to prevent duplicate sends, refreshes, or session changes. A failed message open clears stale details from the reader pane. A failed refresh keeps the previous signed-in state and cached sample.

Gmail read requests use bounded retry with backoff for transient responses. Sends and label changes are not automatically replayed after an error, because that could duplicate mail or repeat a mailbox change.

5 - IMPLEMENTATION

5.15 BUILD AND WINDOWS PACKAGE

The Maven Wrapper builds the project using the JDK version declared in the project. The executable JAR uses ems.App as its production entry point. The demo has a separate launcher so the offline behavior is explicit.

```
.\mvnw.cmd --batch-mode clean package
.\scripts\run-app.ps1 -NoBuild --check-setup
.\scripts\run-app.ps1 -NoBuild
.\scripts\run-demo.ps1
```

The Windows packaging script creates a Windows x64 app image with jpackage and a bundled Java runtime, then prepares the shareable ZIP. Users should extract the complete ZIP before running Maildesk.exe. A live installation still needs MySQL and the user's Desktop OAuth JSON.

The packaged application does not include a private OAuth client file, local token cache, or database password.

5 - IMPLEMENTATION

5.16 VERIFICATION AND TEST PLAN

AREA	CHECKED BEHAVIOR
OAuth	Loopback callback, state validation, token parsing, timeout and error behavior.
Gmail HTTP	Request responses, parsing, MIME construction, label operations, retry boundaries.
Spam rules	Weights, explanations, threshold, and blocked-sender handling.
JDBC	Schema initialization, account isolation, transactional snapshot replacement, feedback.
Swing contract	Search, mailbox navigation, compose validation, failure state, busy controls.
MySQL	Separate opt-in local check using its own fixture account.

The repository's default verification uses local HTTP fixtures and an isolated test database. Automated tests do not send real email. A live account acceptance walkthrough is separate: configure OAuth and MySQL, connect a test account, refresh, compare the bounded sample, and inspect saved rows in MySQL Workbench.

A successful offline demo proves that the UI can be explored without external setup; it does not prove that real Gmail credentials or the local database are configured.

5 - IMPLEMENTATION

5.17 CURRENT LIMITATIONS AND FUTURE WORK

LIMIT TODAY	PROPOSED FOLLOW-UP
At most 18 recent unique messages are cached.	Add user-controlled pagination after measuring API quotas.
Plain-text composition only.	Add attachment preview, safe download, and richer drafts.
Deterministic rule scoring.	Evaluate tuning with a labeled, consented sample; report false positives.
One active account at a time.	Consider a carefully isolated multi-account workspace.
Manual refresh.	Explore scheduled sync and notifications with explicit controls.
Google app remains configured for testing in the student setup.	Complete applicable consent and OAuth verification before broad public use.

The project does not claim a trained AI model. Any future scoring change should preserve visible evidence, make uncertainty understandable, and keep the mailbox owner in the decision loop. Planned features should be tested with controlled data before being described as implemented.

6 - CONCLUSION

Maildesk combines Java Swing, object-oriented service boundaries, Gmail OAuth and REST access, MySQL, and JDBC in one email-management application. The workspace supports mailbox navigation, local search, message reading, validated plain-text composition, and explicit label actions.

The report's principal project contribution is an explainable second opinion for suspicious messages. The rule score is accompanied by reasons, and a person can correct the result. Gmail's own Spam label stays distinct from Maildesk's local review queue. The bounded 18-message sample makes the classroom demonstration predictable and manageable.

A separate offline demo provides fictional messages for presentation and interface review. The live path requires a correctly configured Desktop OAuth client and local MySQL server. The project clearly identifies limits such as attachments, rich text, scheduled synchronization, machine learning, and production OAuth approval as future work.

Through this project, the team can demonstrate how an object-oriented Java application communicates with an external provider, protects account boundaries in a relational schema, and reports user-visible reasons for a security-sensitive suggestion.

7 - REFERENCES

1. Maildesk source repository, setup guide, architecture documentation, SQL schema, and Windows packaging notes. https://github.com/Sanjay-AI-ML/email-management-work

2. Google for Developers. Gmail API documentation. https://developers.google.com/workspace/gmail/api

3. Google Identity. OAuth 2.0 for native and desktop applications. https://developers.google.com/identity/protocols/oauth2/native-app

4. Google for Developers. Gmail API authorization scopes. https://developers.google.com/workspace/gmail/api/auth/scopes

5. Oracle. Java SE documentation. https://docs.oracle.com/en/java/

6. MySQL. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

7. Project source files: src/main/java/ems, src/main/java/com/emailapp/spam, src/demo/java, sql/schema.sql, docs/SETUP.md, and docs/ARCHITECTURE.md.

8. User-provided reference document: EMPLOYEE DETAILS AND PAYROLL MANAGEMENT SYSTEM (academic report structure used as the formatting model).